

Key feedback loops: pose covariance inflates map costs; NCS caps speed; localization failure triggers the Recovery Manager which overrides the planner; planning failure triggers replan, then rotate-search, then safe stop.

3. Risk-Aware Traversability Engine

Principle: invert the problem. Do not detect obstacles and treat everything else as free. Estimate confidently-drivable space and treat everything else as risky. This handles unknown obstacles and ditches without training on them.

Pipeline per frame

- 1. Segmentation gives per-pixel class probabilities p_k .
- 2. Expected traversability: $T = \sum_k (p_k * t_k)$, where t_k is a hand-set score per class (trail 1.0, grass 0.7, gravel 0.6, bush 0.2, obstacle 0, water/ditch 0).
- 3. Uncertainty: U = normalized entropy of p at each pixel.
- 4. Project pixels to the ground plane (inverse perspective mapping, known camera height + pitch) into a BEV grid, e.g. 20 m x 20 m at 0.2 m resolution.
- 5. Geometry term G : in sim, roughness / step height from stereo depth. On real video, relative depth is a cross-check only, not metric.
- 6. Cell risk: $R = (1 - T) + \lambda_u U + \lambda_g G + \lambda_{prox} * (\text{inflation from nearby high-risk cells})$.
- 7. Temporal fusion: ego-motion-compensate the previous map, update in log-odds with decay (filters flicker, remembers cells that left the view).
- 8. Pose-uncertainty inflation: inflate costs proportionally to pose covariance (poor localization = more conservative planning).
- 9. Bands: green $R < 0.25$, yellow < 0.5 , orange < 0.75 , red ≥ 0.75 or lethal (red = forbidden to planner).

Honest caveat: IPM assumes locally flat ground and distorts on slopes. Mitigate with IMU pitch/roll compensation and by weighting far-range cells lower. State this limitation to judges before they find it.

4. GPS-Denied Localization

Option	Verdict
Camera-only monocular VO/SLAM	Scale ambiguity, drifts. Keep as an ablation, not the main path.
Stereo VO/SLAM	Primary in sim: metric scale, robust.
RGB-D	Fails in outdoor sunlight. Drop.
Learned visual localization	Too heavy to train and validate. Drop.
Optical flow	Use only as a cheap health check (motion consistency).
IMU + wheel odometry (simulated)	Fuse via EKF to bridge short visual gaps.

Choice: stereo visual odometry with loop-closure relocalization (RTAB-Map or ORB-SLAM3, whichever builds first on your ROS 2 distro), fused with simulated IMU and wheel odometry in an EKF (robot_localization). **Time-box SLAM integration to one week.** Fallback: own OpenCV pipeline (ORB features, PnP, RANSAC) plus EKF.

Camera-primary honesty: the PS says camera is the primary sensor. IMU/wheel only bridge short visual dropouts. Run a camera-only ablation and report the drift difference openly.

Localization confidence from: tracked feature count vs. expected, RANSAC inlier ratio, reprojection error, EKF covariance trace, and input quality (blur via Laplacian variance, brightness histogram, feature distribution).

Degradation behavior: confidence falls -> slow -> stop -> relocalize (rotate slowly, then retrace toward last good keyframe) -> if recovered resume; if not, dead-reckon on IMU/wheel with growing covariance (planner inflates costs) and a timeout forces safe stop.

5. Perception Model Choices

Task	Choice	Reason
Terrain segmentation (core)	SegFormer-B0 fine-tuned, exported to ONNX (CPU-only fallback: PIDNet-S or BiSeNetV2)	Small (~3.7M params), easy HuggingFace fine-tuning, good accuracy per parameter

Task	Choice	Reason
Dynamic objects (people, animals, vehicles)	YOLOv8n / YOLO11n	Only for moving-object tracking; segmentation handles static terrain
Depth (optional)	Depth Anything V2 Small	Advanced tier: relative depth cross-check on real video only

Rejected: instance segmentation (not needed), larger transformers (latency), TensorRT (only if FPS demands it and an NVIDIA GPU exists). Benchmark FPS and latency on your own hardware; do not quote paper numbers.

Classes (6): smooth-drivable, rough-drivable, vegetation/non-traversable, hard obstacle (rock/log/pole/vehicle/person), water/ditch/negative, background/sky. Ditches are the hardest class: expect poor recall and let the uncertainty and geometry terms cover the gap.

6. Multi-Level Navigation

Level	Algorithm	Why
Mission	D* Lite on a coarse global cost grid	Incremental replanning when the map changes; cheaper than rerunning A*
Local	DWA scoring trajectories on the risk map (goal progress + risk + smoothness + clearance)	Simple, explainable, tunable in days
Reactive	TTC safety layer (Section 7)	Overrides everything

Rejected: RRT* (jerky, sampling variance), TEB and MPC (tuning burden, hard to explain), Hybrid A* (unneeded for differential drive in mostly open terrain).

ROS 2 use: transport, sim bridge and rosbag only. Write the planners yourselves rather than wrestling Nav2 configuration. Use a Nav2 or naive A* stack as the baseline.

Commands: planner outputs (v, w) -> differential drive: $v_{\text{left}} = v - w \cdot L/2$, $v_{\text{right}} = v + w \cdot L/2$, clamped to motor limits. Show it in the dashboard wheel-command panel.

7. Predictive Collision Avoidance

- Predict vehicle trajectory for 3 s at 0.1 s steps from current (v, w).
- Predict moving obstacles with a constant-velocity model from tracked detections (no learned predictors).
- Check vehicle footprint against high-risk cells and predicted obstacle positions at each step. TTC = first step with overlap.
- Stopping time: $\tau_{\text{stop}} = t_{\text{latency}} + v / a_{\text{max}}$ (measure real pipeline latency).
- Collision Risk Score = $\max(\tau_{\text{stop}} / \text{TTC}, \text{mean terrain risk along path})$, clipped to [0, 1].

Condition	Action
$\text{TTC} > 3 \cdot \tau_{\text{stop}}$	Continue
$1.5 \cdot \tau_{\text{stop}} < \text{TTC} \leq 3 \cdot \tau_{\text{stop}}$	Slow proportionally and replan
$\tau_{\text{stop}} < \text{TTC} \leq 1.5 \cdot \tau_{\text{stop}}$	Hard brake and replan / evade
$\text{TTC} \leq \tau_{\text{stop}}$	Emergency stop

Thresholds are starting values. Tune them from seeded test runs and report the final values.

8. Confidence Engine and Cut List

Navigation Confidence Score (NCS) = weakest-link (min, or low-percentile blend) of: perception confidence (mean max-probability in the path corridor), localization confidence, path confidence (cost margin vs. alternatives), input quality, and depth/geometry agreement. Use min, not average: an average lets one failed subsystem hide behind healthy ones.

NCS	State	Behavior
> 0.7	NORMAL	Full speed cap
0.4 - 0.7	CAUTION	Speed cap scales down, wider inflation

NCS	State	Behavior
0.2 - 0.4	RECOVER	Stop/creep, relocalize, re-observe
< 0.2	SAFE_STOP	Hold, alert on dashboard

Add hysteresis (different thresholds for entering vs. leaving states) to prevent oscillation.

Cut list (do not build): Isaac Sim, CARLA, TensorRT (unless needed), learned visual localization, RRT*/TEB/MPC, instance segmentation, learned obstacle-motion prediction, Three.js 3D (use a 2D canvas dashboard), transformer detectors, full domain-adaptation research, and the physical UGV (only if all core milestones are done with time to spare).

9. Autonomous Recovery Manager

Failure	Detection	Response	Resume
Poor light / shadows	Brightness histogram, segmentation entropy spike	Slow, inflate costs, lean on IMU/wheel	NCS above threshold
Motion blur	Laplacian variance drop	Reduce speed until sharp	Sharpness recovers
Camera obstruction	Low image variance, feature collapse	Stop, flag	Frame variance returns
Featureless terrain	Tracked features below minimum	Slow, dead-reckon with covariance growth	Features return
Localization failure	Covariance / inlier collapse	Stop, relocalize, retrace to last keyframe	Confidence restored
Sudden obstacle	TTC drop	Section 7 response	Path clear
Input loss	Frame timeout	Stop after grace period	Frames return
Planning failure	No path found	Rotate-search, back up, replan	Path found
Dead end	Repeated replans, no progress	Backtrack to last junction, mark region blocked	New route found

Rule: every state transition writes a reason to the decision log. No silent failures.

10. Simulation Strategy

The honest problem: a model trained on real datasets (RUGD, RELIS-3D) will not transfer to Gazebo's synthetic graphics. Use two coordinated tracks:

- **Track A - closed-loop sim:** Gazebo Harmonic + ROS 2 Jazzy (Ubuntu 24.04; or Webots if the team cannot run Linux natively). World with trail, rocks, trees, a ditch, slopes, a dynamic obstacle and lighting variants. Use simulator free labels to generate synthetic segmentation data; fine-tune on sim + augmentations so the closed loop works.
- **Track B - real-video perception + VO:** run the same perception, risk map and confidence engine on real outdoor footage (RELIS/RUGD sequences plus own campus clips). This is open-loop; label it honestly.
- **Scenario generator:** parameterized worlds with seeds (obstacle positions, lighting, start/goal) so every test is reproducible.

11. Dashboard and Explainability

React + TypeScript, 2D canvas/SVG, FastAPI + WebSocket. Panels:

- Camera feed with segmentation overlay.
- Top-down map: risk map, planned path, actual path, UGV pose, detected obstacles.
- Gauges: NCS and its components, localization confidence, speed, TTC.
- State machine indicator with e-stop status.
- Explanation feed (structured), e.g. 't=42.1s CAUTION: localization confidence 0.41 (features 38 -> 12). Speed limit 0.8 -> 0.3 m/s.'
- Replan event markers on the path.
- Wheel command panel (v, w, left/right).

- Fault injection panel: spawn obstacle, blur, darken, cover camera (makes the demo judge-proof).
- Side-by-side mode: baseline vs. VISTRA on the same seed.

12. The Three Innovations

A. Uncertainty-propagating traversability mapping

- Problem: argmax segmentation plus hard obstacle boxes hides model doubt and misses unknown hazards.
- Approach: probabilistic cost from class posteriors plus entropy, log-odds temporal fusion, pose-covariance inflation, free-space-first formulation.
- Advantage: unknown obstacles and shaky perception become costly rather than invisible.
- Measure: collision rate and near-miss distance on unknown-obstacle and poor-lighting scenarios vs. baseline; ablation with entropy term off.

B. Confidence-gated behavior governor

- Problem: most stacks keep driving at full speed with a degrading pose.
- Approach: weakest-link NCS with a hysteresis FSM controlling speed cap and recovery.
- Measure: recovery success rate and collisions during injected degradation; ablation with governor off.

C. TTC + terrain-risk predictive safety layer

- Problem: 'obstacle detected -> turn' ignores speed, latency and stopping distance.
- Approach: Section 7, with measured latency inside τ_{stop} .
- Measure: minimum clearance, emergency-stop rate, false-stop rate vs. a fixed-distance-threshold baseline.

Together: **uncertainty in -> safe behavior out.**

13. Dataset Strategy

- **Public:** RUGD, RELIS-3D, Freiburg Forest, GOOSE if accessible. Map labels down to the 6 classes.
- **Synthetic:** auto-labelled sim frames across lighting and weather variants.
- **Self-collected:** about 300-500 campus/trail frames (phone), annotated with CVAT or Label Studio; used mainly for validation and the real-video demo.
- **Hard negatives:** shadows on trails, puddles, tall grass hiding rocks, low-sun glare.
- **Splits:** by sequence or location, never random frames. Keep a held-out real set that never enters training.
- **Metrics:** per-class IoU (report ditch/obstacle separately), plus FPS and latency on your hardware.
- **Augmentation:** brightness/contrast, blur, shadows, color jitter, random occlusion patches.

14. Software Stack and Hardware Abstraction

Python, PyTorch + HuggingFace Transformers (SegFormer), ONNX Runtime, OpenCV, NumPy, ROS 2 Jazzy, Gazebo Harmonic, robot_localization, RTAB-Map or ORB-SLAM3, FastAPI + WebSocket, React + TypeScript with canvas. Optional later: TensorRT.

Abstraction: CameraInterface, VehicleInterface, SensorInterface with implementations SimCamera, VideoFileCamera, WebcamCamera, SimVehicle, RealVehicle (optional). The core never imports a hardware-specific module, which gives four modes (sim / recorded / webcam / physical) without touching navigation code.

15. Testing and Baseline Benchmark

Baseline (build early): YOLOv8 detection + the same VO/SLAM + A* on a binary occupancy grid + fixed-distance stop rule, in the same simulator, with the same vehicle and seeds.

Scenario	What it tests
Normal terrain	Sanity / efficiency
Difficult terrain	Risk map, slopes, rough surfaces

Scenario	What it tests
Sudden obstacle	TTC layer, replanning
Moving obstacle	Constant-velocity prediction
Poor lighting	Perception confidence, governor
Localization degradation	Recovery Manager
Perception failure	Fail-safe behavior
Narrow path	Clearance, path width
Dead end	Backtrack logic
Multiple routes	Risk-aware route choice
Unknown obstacle	Free-space-first formulation

Protocol: 20-30 seeded runs per scenario per system; report mean and confidence intervals, not single runs.

Metrics: success, collisions, path length, time, localization error (vs. sim ground truth), path deviation, replans, FPS, latency, NCS trace, recovery time.

Ablations: full VISTRA minus each of A, B and C. Report results honestly; if the baseline wins a scenario, say so and explain why. Do not fabricate numbers.

16. PS Requirement Mapping

PS requirement	Module	Evidence	Demo
Path detection	SegFormer + risk map	Per-class IoU, FPS/latency	Overlay + BEV map, real-video track
Visual localization	Stereo VO + EKF + Recovery Manager	ATE vs. ground truth, drift ablation	Blur/dark injection with pose trace
Collision avoidance	D* Lite + DWA + TTC layer	Collision rate, min clearance vs. baseline	Sudden obstacle
Wheel commands	(v, w) to wheel mapper	Command log	Wheel panel
Collision-free A to B	Full stack	Scenario suite results	Complete run

17. Demo Script and Backup

- 1. Start with no GPS, camera feed and an empty map.
- 2. Segmentation and the risk map build up on the dashboard.
- 3. UGV starts toward B; global path shown.
- 4. Judge clicks 'spawn obstacle'. TTC and risk rise, speed drops, replan triggers, new path appears.
- 5. UGV follows the new path.
- 6. Judge clicks 'darken/blur camera'. NCS drops, CAUTION then RECOVER; it relocalizes and resumes.
- 7. Reaches B. Summary panel: collisions, replans, recovery time.
- 8. Play the side-by-side baseline run on the same seed.

Backup: rosbag replays of every scenario with fixed seeds, a pre-rendered video of the full run, a recorded-video mode for perception, and a playback dashboard mode. Test the backup at least three times before the finale.

18. Risk Register and Feature Tiers

Risk	Prob.	Impact	Mitigation / backup
SLAM integration eats weeks	High	High	1-week time-box; own OpenCV VO + EKF fallback
Sim-to-real perception gap	High	Medium	Fine-tune on sim data; separate real-video track
Ditch / negative-obstacle detection weak	High	Medium	Uncertainty + geometry terms; be honest in slides
Live sim crashes at demo	Medium	High	Rosbag replay + video backup
Gazebo/ROS setup pain on laptops	Medium	Medium	One Docker/Ubuntu image for the team; Webots fallback

Risk	Prob.	Impact	Mitigation / backup
Latency too high for TTC claims	Medium	Medium	Lower resolution, ONNX Runtime, measure real tau_stop
Scope creep	Very high	High	Freeze scope at end of Phase 6
Overfitting / inflated metrics	Medium	High	Sequence-level splits; held-out real set

Must-have: sim + vehicle interface, segmentation, BEV risk map, VO + EKF, D* Lite + DWA, TTC layer, dashboard, baseline benchmark.

Important: confidence engine, recovery manager, explanation feed, fault injector.

Advanced: relative-depth cross-check, moving-obstacle prediction, live side-by-side mode.

Optional: physical UGV, TensorRT.

19. Development Roadmap (assume 8-10 weeks; compress if shorter)

Phase	Weeks	Definition of Done
1. Sim + virtual UGV	1	Vehicle drivable by teleop, camera/IMU topics streaming, scenarios seeded
2. Perception	1-3	SegFormer-B0 ONNX, real and sim IoU reported, FPS measured
3. Traversability	2-3	BEV risk map with bands, temporal fusion working
4. Localization	2-4	VO + EKF; ATE vs. ground truth logged; time-boxed
5-6. Mapping / planning	3-5	D* Lite + DWA drive A to B on a clear world
7. Collision avoidance	5-6	TTC layer stops before every injected obstacle across 20 seeds
8. Confidence engine	6	NCS gates speed; hysteresis verified
9. Recovery manager	6-7	Recovers from injected blur/dark/loss in the test suite
10. Dashboard	3-8 (parallel)	All panels live from the WebSocket
11. Benchmarking	7-9	Baseline vs. VISTRA on all scenarios, ablations done
12. Final demo	9-10	Three full rehearsals including backup mode

Freeze features at end of week 6. Weeks 7-10 are for testing, benchmarking and rehearsal only.

20. Team Roles (5 people)

- **Perception lead:** datasets, SegFormer training, ONNX, class mapping.
- **Localization lead:** VO/SLAM, EKF, confidence signals, relocalization.
- **Planning/safety lead:** risk map fusion, D* Lite, DWA, TTC layer, FSM.
- **Sim/integration lead:** Gazebo worlds, scenario generator, ROS 2 bridge, Docker image, rosbag backup.
- **Dashboard + benchmark lead:** React UI, FastAPI, test harness, metrics and plots, baseline.
- A sixth person: baseline, documentation and presentation.

21. Judge Questions to Rehearse

Question	Defensible answer
Isn't this just YOLO + SLAM + A*?	No. That is our baseline, and we benchmark against it. Ours differs in uncertainty-propagating cost mapping, confidence-gated behavior and TTC-based safety, each with an ablation.
You use IMU and wheel odometry. Is it really camera-based?	Vision is the primary localization signal and the sole perception sensor. IMU/wheel only bridge short visual dropouts. We show a camera-only ablation and its drift.
Does it work in the real world?	Perception and mapping run on real footage (open-loop). Closed-loop navigation is validated in simulation. Physical UGV is future validation.
How do you handle ditches?	Poorly if relying on appearance alone. Uncertain terrain is costly by default and the geometry term adds a second signal. We report ditch recall separately.
Why not end-to-end learned planning?	Explainability, verifiability, low data needs, and a safety layer you can reason about.
How did you tune thresholds?	From seeded test runs; final values and sensitivity are reported.

Question	Defensible answer
What is your latency budget?	Measured end to end and fed into stopping time.
What are your limitations?	Flat-ground IPM assumption, sim-to-real gap, ditch recall, hand-set class costs. State them before being asked.

22. Priorities Summary

- Win on evidence and reliability, not feature count.
- Strongest differentiators: judge-controlled fault injector, side-by-side baseline run, reproducible scenario suite with confidence intervals.
- Build the baseline and the benchmark harness in weeks 1-3, not at the end.
- Decide the SLAM library and the simulator immediately and do not revisit either.
- Freeze scope at the end of week 6.